

Chapter 3. RAG Part II: Chatting with Your Data

In the previous chapter, you learned how to process your data and create and store embeddings in a vector store. In this chapter, you'll learn how to efficiently retrieve the most relevant embeddings and chunks of documents based on a user's query. This enables you to construct a prompt that contains relevant documents as context, improving the accuracy of the LLM's final output.

This process—which involves embedding a user's query, retrieving similar documents from a data source, and then passing them as context to the prompt sent to the LLM—is formally known as *retrieval-augmented generation* (RAG).

RAG is an essential component of building chat-enabled LLM apps that are accurate, efficient, and up-to-date. In this chapter, you'll progress from basics to advanced strategies to build an effective RAG system for various data sources (such as vector stores and databases) and data structures (structured and unstructured).

But first, let's define RAG and discuss its benefits.

Introducing Retrieval-Augmented Generation

RAG is a technique used to enhance the accuracy of outputs generated by LLMs by providing context from external sources. The term was originally coined in a paper by Meta AI researchers who discovered that RAG-enabled models are more factual and specific than non-RAG models.¹

Without RAG, the LLM relies solely on its pretrained data, which may be outdated. For example, let's ask ChatGPT a question about a current event and see its response:

Input

```
Which country is the latest winner of the men's FIFA World Cup?
```

Output

```
The most recent FIFA World Cup winner was France, who won the tournament in
```

The response by the LLM is factually incorrect and outdated. The latest winner at the time of this book's publication is Argentina, who won the World Cup in 2022. While this example question may be trivial, LLM hallucination can have disastrous consequences if its answers are relied upon for fact-checking or important decision making.

To combat this problem, we need to provide the LLM with factual, up-to-date information from which it can formulate an accurate response. Continuing on from the previous example, let's go over to Wikipedia's page for the [FIFA World Cup](#), copy the introduction paragraph, and then append it as *context* to our prompt to ChatGPT:

```
Which country is the latest winner of the men's FIFA World Cup?
```

```
See context below.
```

```
The FIFA World Cup, often called the World Cup, is an international association football competition among the senior men's national teams of the members of the Fédération Internationale de Football Association (FIFA), the sport's global governing body. The tournament has been held every four years since its inaugural tournament in 1930, with the exception of 1942 and 1946 due to the Second World War. The reigning champions are Argentina, who won their third title at the 2022 tournament.
```

Note that the last sentence contains the necessary context the LLM can use to provide an accurate answer. Here's the response from the LLM:

```
The latest winner of the men's FIFA World Cup is Argentina, who won their third title at the 2022 tournament.
```

Because of the up-to-date additional context provided, the LLM was able to generate an accurate response to the prompt. But copying and pasting relevant information as context isn't practical nor scalable for a production AI application. We need an automated system to fetch relevant information based on a user's query, append it as context to the prompt, and then execute the generation request to the LLM.

Retrieving Relevant Documents

A RAG system for an AI app typically follows three core stages:

Indexing

This stage involves preprocessing the external data source and storing embeddings that represent the data in a vector store where they can be easily retrieved.

Retrieval

This stage involves retrieving the relevant embeddings and data stored in the vector store based on a user's query.

Generation

This stage involves synthesizing the original prompt with the retrieved relevant documents as one final prompt sent to the model for a prediction.

The three basic stages look like [Figure 3-1](#).

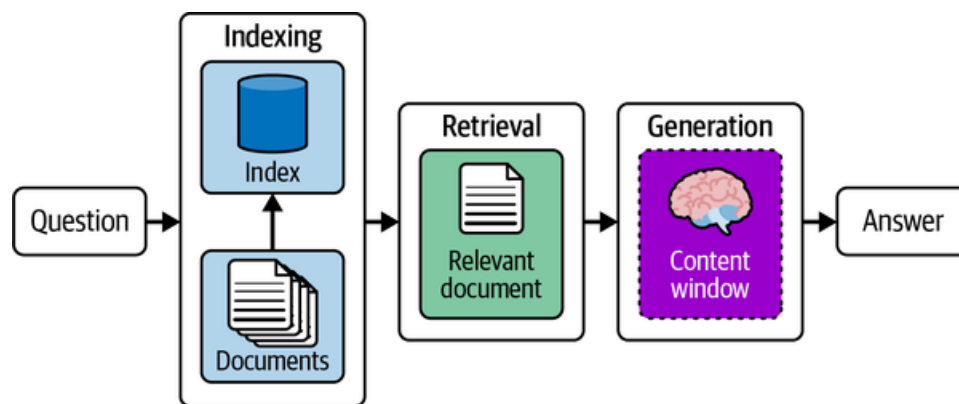


Figure 3-1. The key stages of RAG

The indexing stage of this process was covered extensively in [Chapter 2](#), where you learned how to use document loaders, text splitters, embeddings, and vector stores.

Let's run through an example from scratch again, starting with the indexing stage:

Python

```
from langchain_community.document_loaders import TextLoader
from langchain_openai import OpenAIEmbeddings
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_postgres.vectorstores import PGVector

# Load the document, split it into chunks
raw_documents = TextLoader('./test.txt').load()
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000,
                                              chunk_overlap=200)
documents = text_splitter.split_documents(raw_documents)

# embed each chunk and insert it into the vector store
model = OpenAIEmbeddings()
```

```
connection = 'postgresql+psycopg://langchain:langchain@localhost:6024/langc
db = PGVector.from_documents(documents, model, connection=connection)
```

JavaScript

```
import { TextLoader } from "langchain/document_loaders/fs/text";
import { RecursiveCharacterTextSplitter } from "@langchain/textsplitters";
import { OpenAIEmbeddings } from "@langchain/openai";
import { PGVectorStore } from "@langchain/community/vectorstores/pgvector";

// Load the document, split it into chunks
const loader = new TextLoader("./test.txt");
const raw_docs = await loader.load();
const splitter = new RecursiveCharacterTextSplitter({
  chunkSize: 1000,
  chunkOverlap: 200,
});
const docs = await splitter.splitDocuments(raw_docs)

// embed each chunk and insert it into the vector store
const model = new OpenAIEmbeddings();
const db = await PGVectorStore.fromDocuments(docs, model, {
  postgresConnectionOptions: {
    connectionString: 'postgresql://langchain:langchain@localhost:6024/langc
  }
})
```

[Chapter 2](#) has more details on the indexing stage.

The indexing stage is now complete. In order to execute the retrieval stage, we need to perform similarity search calculations—such as cosine similarity—between the user’s query and our stored embeddings, so relevant chunks of our indexed document are retrieved (see [Figure 3-2](#)).

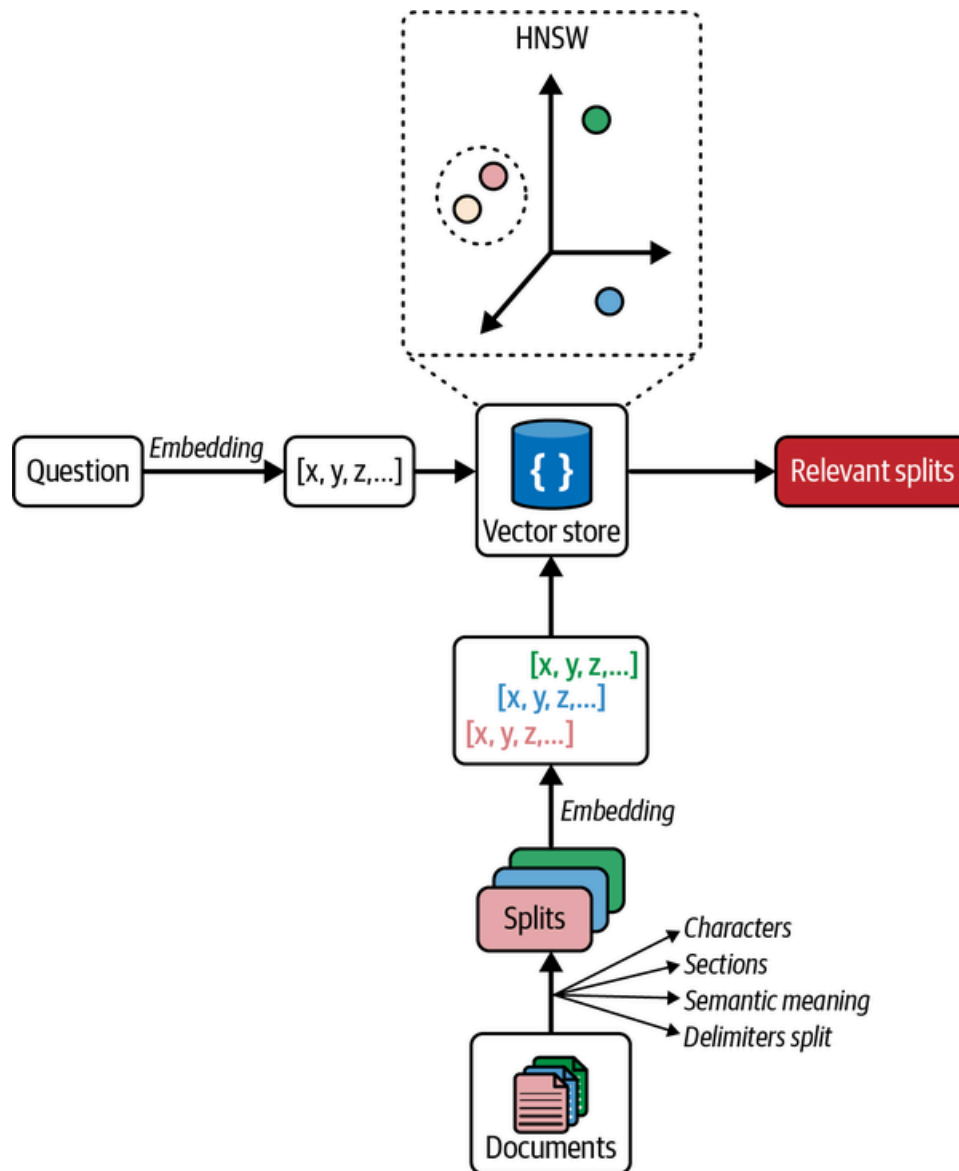


Figure 3-2. An example flow of indexing documents alongside retrieval of relevant documents from a vector store; the Hierarchical Navigable Small World (HNSW) box depicts calculating similarity of documents against the user's query

[Figure 3-2](#) illustrates the steps in the retrieval process:

1. Convert the user's query into embeddings.
2. Calculate the embeddings in the vector store that are most similar to the user's query.
3. Retrieve the relevant document embeddings and their corresponding text chunk.

We can represent these steps programmatically using LangChain as follows:

Python

```
# create retriever
retriever = db.as_retriever()

# fetch relevant documents
docs = retriever.invoke("""Who are the key figures in the ancient greek
history of philosophy?""")
```

JavaScript

```
// create retriever
const retriever = db.asRetriever()

// fetch relevant documents
const docs = await retriever.invoke(`Who are the key figures in the ancient
greek history of philosophy?`)
```

Note that we are using a vector store method you haven't seen before:

`as_retriever`. This function abstracts the logic of embedding the user's query and the underlying similarity search calculations performed by the vector store to retrieve the relevant documents.

There is also an argument `k`, which determines the number of relevant documents to fetch from the vector store. For example:

Python

```
# create retriever with k=2
retriever = db.as_retriever(search_kwargs={"k": 2})

# fetch the 2 most relevant documents
docs = retriever.invoke("""Who are the key figures in the ancient greek his
of philosophy?""")
```

JavaScript

```
// create retriever with k=2
const retriever = db.asRetriever({k: 2})

// fetch the 2 most relevant documents
const docs = await retriever.invoke(`Who are the key figures in the ancient
greek history of philosophy?`)
```

In this example, the argument `k` is specified as 2. This tells the vector store to return the two most relevant documents based on the user's query.

It may seem counterintuitive to use a low `k` value, but retrieving more documents is not always better. The more documents are retrieved, the slower your application will perform, the larger the prompt (and associated cost of generation) will be, and the greater the likelihood of retrieving chunks of text that contain irrelevant information, which will cause the LLM to hallucinate.

Now that we've completed the retrieval stage of the RAG system, let's move on to the final generation stage.

Generating LLM Predictions Using Relevant Documents

Once we've retrieved the relevant documents based on the user's query, the final step is to add them to the original prompt as context and then invoke the model to generate a final output ([Figure 3-3](#)).

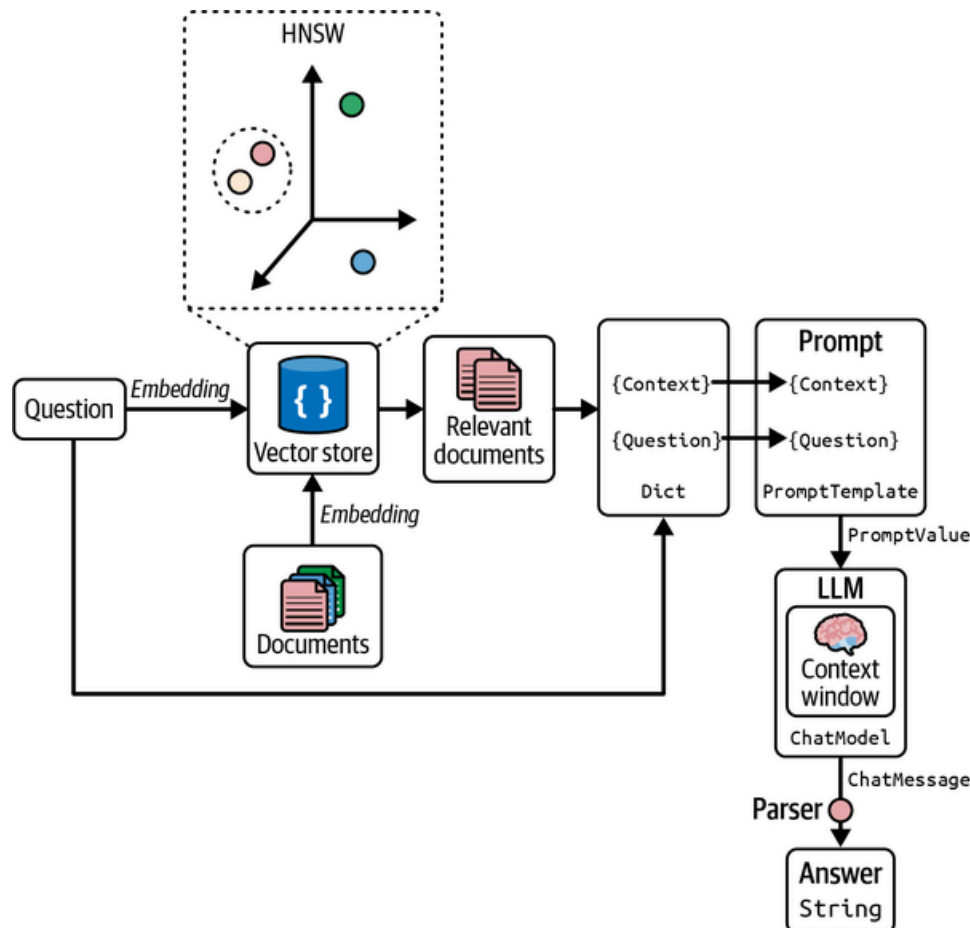


Figure 3-3. An example flow demonstrating indexing documents, retrieval of relevant documents from a vector store, and inclusion of retrieved documents as context in the LLM prompt

Here's a code example continuing on from our previous example:

Python

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate

retriever = db.as_retriever()

prompt = ChatPromptTemplate.from_template("""Answer the question based only
the following context:
{context}

Question: {question}
""")
```

```

llm = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0)

chain = prompt | llm

# fetch relevant documents
docs = retriever.get_relevant_documents("""Who are the key figures in the
ancient greek history of philosophy?""")

# run
chain.invoke({"context": docs, "question": """Who are the key figures in the
ancient greek history of philosophy?"""})

```

JavaScript

```

import {ChatOpenAI} from '@langchain/openai'
import {ChatPromptTemplate} from '@langchain/core/prompts'

const retriever = db.asRetriever()

const prompt = ChatPromptTemplate.fromTemplate(`Answer the question based on
the following context:
{context}

Question: {question}
`)

const llm = new ChatOpenAI({temperature: 0, modelName: 'gpt-3.5-turbo'})

const chain = prompt.pipe(llm)

// fetch relevant documents
const docs = await retriever.invoke(`Who are the key figures in the ancient
greek history of philosophy?`)

await chain.invoke({context: docs, question: `Who are the key figures in th
ancient greek history of philosophy?`})

```

Note the following changes:

- We implement dynamic `context` and `question` variables into our prompt, which allows us to define a `ChatPromptTemplate` the model can use to generate a response.
- We define a `ChatOpenAI` interface to act as our LLM. Temperature is set to 0 to eliminate the creativity in outputs from the model.
- We create a chain to compose the prompt and LLM. A reminder: the `|` operator (or `pipe` method in JS) takes the output of `prompt` and uses it as the input to `llm`.
- We `invoke` the chain passing in the `context` variable (our retrieved relevant docs) and the user's question to generate a final output.

We can encapsulate this retrieval logic in a single function:

Python

```
from langchain_openai import ChatOpenAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import chain

retriever = db.as_retriever()

prompt = ChatPromptTemplate.from_template("""Answer the question based only
the following context:
{context}

Question: {question}
""")

llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)

@chain
def qa(input):
    # fetch relevant documents
    docs = retriever.get_relevant_documents(input)
    # format prompt
    formatted = prompt.invoke({"context": docs, "question": input})
    # generate answer
    answer = llm.invoke(formatted)
    return answer

# run
qa.invoke("Who are the key figures in the ancient greek history of philosop
```

JavaScript

```
import {ChatOpenAI} from '@langchain/openai'
import {ChatPromptTemplate} from '@langchain/core/prompts'
import {RunnableLambda} from '@langchain/core/runnables'

const retriever = db.asRetriever()

const prompt = ChatPromptTemplate.fromTemplate(`Answer the question based o
on the following context:
{context}

Question: {question}
`)

const llm = new ChatOpenAI({temperature: 0, modelName: 'gpt-3.5-turbo'})

const qa = RunnableLambda.from(async input => {
  // fetch relevant documents
  const docs = await retriever.invoke(input)
```

```
// format prompt
const formatted = await prompt.invoke({context: docs, question: input})
// generate answer
const answer = await llm.invoke(formatted)
return answer
})

await qa.invoke(`Who are the key figures in the ancient greek history of
philosophy?`)
```

Notice how we now have a new runnable `qa` function that can be called with just a question and takes care to first fetch the relevant docs for context, format them into the prompt, and finally generate the answer. In the Python code, the `@chain` decorator turns the function into a runnable chain. This notion of encapsulating multiple steps into a single function will be key to building interesting apps with LLMs.

You can also return the retrieved documents for further inspection:

Python

```
@chain
def qa(input):
    # fetch relevant documents
    docs = retriever.get_relevant_documents(input)
    # format prompt
    formatted = prompt.invoke({"context": docs, "question": input})
    # generate answer
    answer = llm.invoke(formatted)
    return {"answer": answer, "docs": docs}
```

JavaScript

```
const qa = RunnableLambda.from(async input => {
  // fetch relevant documents
  const docs = await retriever.invoke(input)
  // format prompt
  const formatted = await prompt.invoke({context: docs, question: input})
  // generate answer
  const answer = await llm.invoke(formatted)
  return {answer, docs}
})
```

Congratulations! You've now built a basic RAG system to power an AI app for personal use.

However, a production-ready AI app used by multiple users requires a more advanced RAG system. In order to build a robust RAG system, we

need to answer the following questions effectively:

- How do we handle the variability in the quality of a user's input?
- How do we route queries to retrieve relevant data from a variety of data sources?
- How do we transform natural language to the query language of the target data source?
- How do we optimize our indexing process, i.e., embedding, text splitting?

Next we'll discuss the latest research-backed strategies to answer these questions and build a production-ready RAG system. These strategies can be summarized in [Figure 3-4](#).

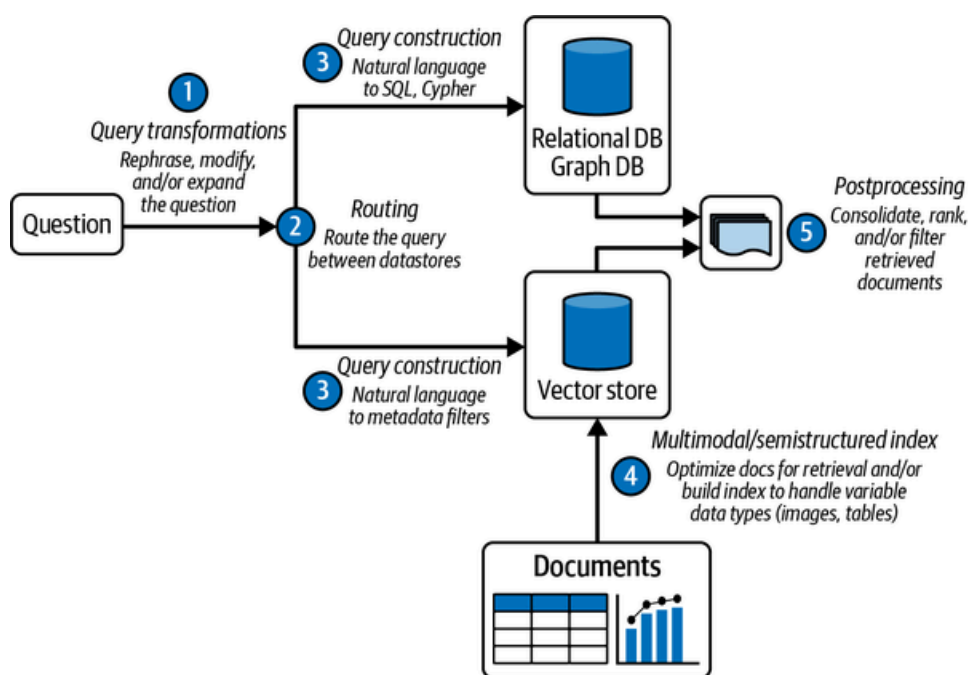


Figure 3-4. Effective strategies to optimize the accuracy of your RAG system

NOTE

All code blocks in the rest of this chapter use the vector store we set up at the beginning of the chapter.

Query Transformation

One of the major problems with a basic RAG system is that it relies too heavily on the quality of a user's query to generate an accurate output. In a production setting, a user is likely to construct their query in an incomplete, ambiguous, or poorly worded manner that leads to model hallucination.

Query transformation is a subset of strategies designed to modify the user's input to answer the first RAG problem question: How do we handle

the variability in the quality of a user's input? [Figure 3-5](#) illustrates the range of query transformation strategies, ranging from those that make a user's input more or less abstract in order to generate an accurate LLM output. The next section begins with a middle ground strategy.

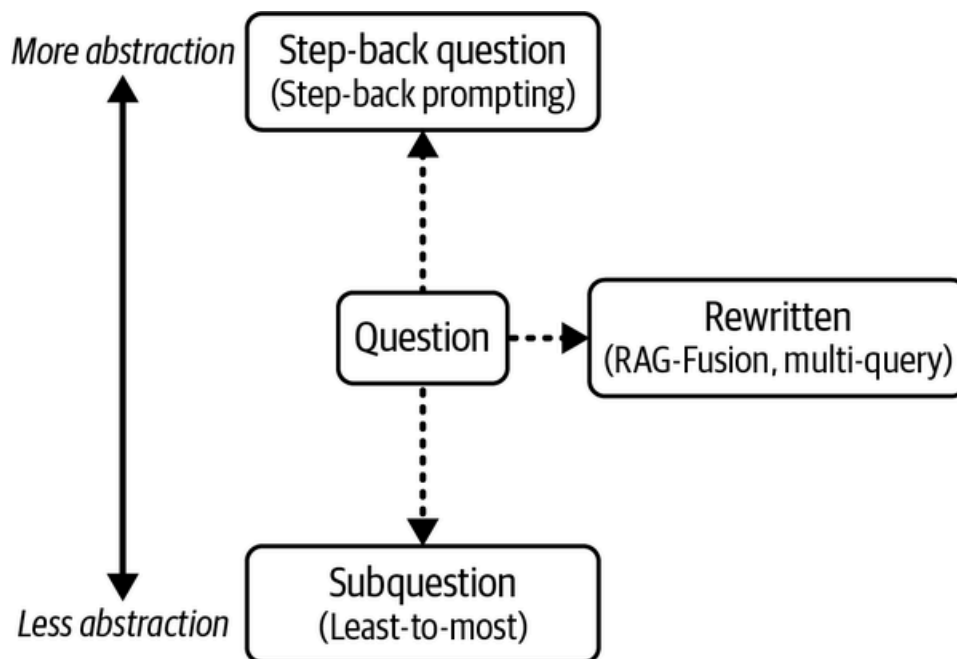


Figure 3-5. Various methods to transform a user's query based on the abstraction level

Rewrite-Retrieve-Read

The Rewrite-Retrieve-Read strategy proposed by a Microsoft Research team simply prompts the LLM to rewrite the user's query before performing retrieval.² To illustrate, let's return to the chain we built in the previous section, this time invoked with a poorly worded user query:

Python

```
@chain
def qa(input):
    # fetch relevant documents
    docs = retriever.get_relevant_documents(input)
    # format prompt
    formatted = prompt.invoke({"context": docs, "question": input})
    # generate answer
    answer = llm.invoke(formatted)
    return answer

qa.invoke("""Today I woke up and brushed my teeth, then I sat down to read
news. But then I forgot the food on the cooker. Who are some key figure
the ancient greek history of philosophy?""")
```

JavaScript

```

const qa = RunnableLambda.from(async input => {
  // fetch relevant documents
  const docs = await retriever.invoke(input)
  // format prompt
  const formatted = await prompt.invoke({context: docs, question: input})
  // generate answer
  const answer = await llm.invoke(formatted)
  return answer
})

await qa.invoke(`Today I woke up and brushed my teeth, then I sat down to read
the news. But then I forgot the food on the cooker. Who are some key figures
in the ancient greek history of philosophy?`)

```

The output (remember: if you rerun it, your output might be different from this):

```
Based on the given context, there is no information provided.
```

The model failed to answer the question because it was distracted by the irrelevant information provided in the user's query.

Now let's implement the Rewrite-Retrieve-Read prompt:

Python

```

rewrite_prompt = ChatPromptTemplate.from_template("""Provide a better search
query for web search engine to answer the given question, end the query
with '***'. Question: {x} Answer: """)

def parse_rewriter_output(message):
    return message.content.strip(' ').strip("***")

rewriter = rewrite_prompt | llm | parse_rewriter_output

@chain
def qa_rrr(input):
    # rewrite the query
    new_query = rewriter.invoke(input)
    # fetch relevant documents
    docs = retriever.get_relevant_documents(new_query)
    # format prompt
    formatted = prompt.invoke({"context": docs, "question": input})
    # generate answer
    answer = llm.invoke(formatted)
    return answer

# run
qa_rrr.invoke("""Today I woke up and brushed my teeth, then I sat down to read
the news. But then I forgot the food on the cooker. Who are some key figures
in the ancient greek history of philosophy?""")

```

```
the news. But then I forgot the food on the cooker. Who are some key figures in the ancient greek history of philosophy?""")
```

JavaScript

```
const rewritePrompt = ChatPromptTemplate.fromTemplate(`Provide a better search query for web search engine to answer the given question, end the queries with '**'. Question: {question} Answer:`)

const rewriter = rewritePrompt.pipe(llm).pipe(message => {
  return message.content.replaceAll('"', ' ').replaceAll('**')
})

const qa = RunnableLambda.from(async input => {
  const newQuery = await rewriter.invoke({question: input});
  // fetch relevant documents
  const docs = await retriever.invoke(newQuery)
  // format prompt
  const formatted = await prompt.invoke({context: docs, question: input})
  // generate answer
  const answer = await llm.invoke(formatted)
  return answer
})

await qa.invoke(`Today I woke up and brushed my teeth, then I sat down to read the news. But then I forgot the food on the cooker. Who are some key figures in the ancient greek history of philosophy?`)
```

The output:

```
Based on the given context, some key figures in the ancient greek history of philosophy include: Themistocles (an Athenian statesman), Pythagoras, and P
```

Notice that we have had an LLM rewrite the user's initial distracted query into a much clearer one, and it is that more focused query that is passed to the retriever to fetch the most relevant documents. Note: this technique can be used with any retrieval method, be that a vector store such as we have here or, for instance, a web search tool. The downside of this approach is that it introduces additional latency into your chain, because now we need to perform two LLM calls in sequence.

Multi-Query Retrieval

A user's single query can be insufficient to capture the full scope of information required to answer the query comprehensively. The multi-query retrieval strategy resolves this problem by instructing an LLM to generate multiple queries based on a user's initial query, executing a parallel re-

retrieval of each query from the data source and then inserting the retrieved results as prompt context to generate a final model output.

[Figure 3-6](#) illustrates.

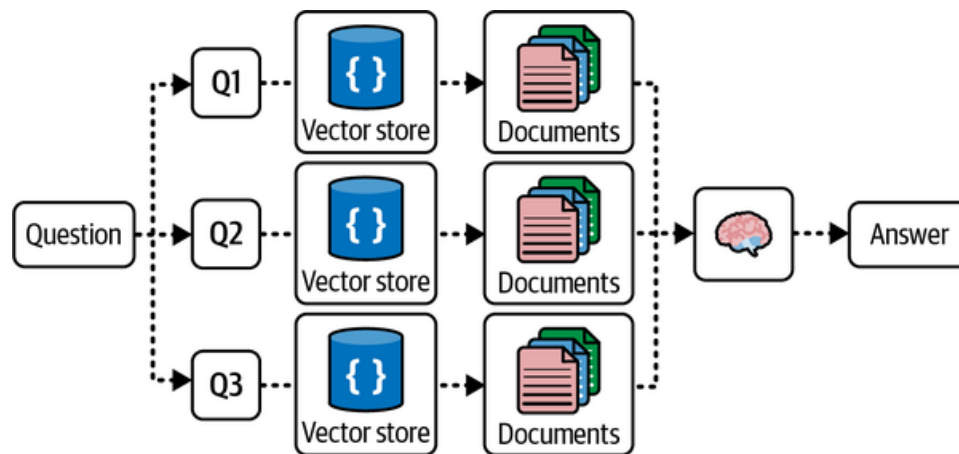


Figure 3-6. Demonstration of the multi-query retrieval strategy

This strategy is particularly useful for use cases where a single question may rely on multiple perspectives to provide a comprehensive answer.

Here's a code example of multi-query retrieval in action:

Python

```
from langchain.prompts import ChatPromptTemplate

perspectives_prompt = ChatPromptTemplate.from_template("""You are an AI language
model assistant. Your task is to generate five different versions of the
given user question to retrieve relevant documents from a vector database.
By generating multiple perspectives on the user question, your goal is to
help the user overcome some of the limitations of the distance-based
similarity search. Provide these alternative questions separated by
newlines. Original question: {question}""")

def parse_queries_output(message):
    return message.content.split('\n')

query_gen = perspectives_prompt | llm | parse_queries_output
```

JavaScript

```
const perspectivesPrompt = ChatPromptTemplate.fromTemplate(`You are an AI
language model assistant. Your task is to generate five different version
of the given user question to retrieve relevant documents from a vector
database. By generating multiple perspectives on the user question, your
goal is to help the user overcome some of the limitations of the
distance-based similarity search. Provide these alternative questions
separated by newlines. Original question: {question}`)

const queryGen = perspectivesPrompt.pipe(llm).pipe(message => {
```

```
    return message.content.split('\n')
  })
```

Note that the prompt template is designed to generate variations of questions based on the user's initial query.

Next we take the list of generated queries, retrieve the most relevant docs for each of them in parallel, and then combine to get the unique union of all the retrieved relevant documents:

Python

```
def get_unique_union(document_lists):
    # Flatten list of lists, and dedupe them
    deduped_docs = {
        doc.page_content: doc
        for sublist in document_lists for doc in sublist
    }
    # return a flat list of unique docs
    return list(deduped_docs.values())

retrieval_chain = query_gen | retriever.batch | get_unique_union
```

JavaScript

```
const retrievalChain = queryGen
  .pipe(retriever.batch.bind(retriever))
  .pipe(documentLists => {
    const dedupedDocs = {}
    documentLists.flat().forEach(doc => {
      dedupedDocs[doc.pageContent] = doc
    })
    return Object.values(dedupedDocs)
  })
```

Because we're retrieving documents from the same retriever with multiple (related) queries, it's likely at least some of them are repeated. Before using them as context to answer the question, we need to deduplicate them, to end up with a single instance of each. Here we dedupe docs by using their content (a string) as the key in a dictionary (or object in JS), because a dictionary can only contain one entry for each key. After we've iterated through all docs, we simply get all the dictionary values, which is now free of duplicates.

Notice our use as well of `.batch`, which runs all generated queries in parallel and returns a list of the results—in this case, a list of lists of documents, which we then flatten and dedupe as described earlier.

This final step is to construct a prompt, including the user's question and combined retrieved relevant documents, and a model interface to generate the prediction:

Python

```
prompt = ChatPromptTemplate.from_template("""Answer the following question
on this context:

{context}

Question: {question}
""")

@chain
def multi_query_qa(input):
    # fetch relevant documents
    docs = retrieval_chain.invoke(input)
    # format prompt
    formatted = prompt.invoke({"context": docs, "question": input})
    # generate answer
    answer = llm.invoke(formatted)
    return answer

# run
multi_query_qa.invoke("""Who are some key figures in the ancient greek history
of philosophy?""")
```

JavaScript

```
const prompt = ChatPromptTemplate.fromTemplate(`Answer the following
question based on this context:

{context}

Question: {question}
`)

const multiQueryQa = RunnableLambda.from(async input => {
    // fetch relevant documents
    const docs = await retrievalChain.invoke(input)
    // format prompt
    const formatted = await prompt.invoke({context: docs, question: input})
    // generate answer
    const answer = await llm.invoke(formatted)
    return answer
})

await multiQueryQa.invoke(`Who are some key figures in the ancient greek
history of philosophy?`)
```

Notice how this isn't that different from our previous QA chains, as all the new logic for multi-query retrieval is contained in `retrieval_chain`. This is key to making good use of these techniques—implementing each technique as a standalone chain (in this case, `retrieval_chain`), which makes it easy to adopt them and even to combine them.

RAG-Fusion

The RAG-Fusion strategy shares similarities with the multi-query retrieval strategy, except we will apply a final reranking step to all the retrieved documents.³ This reranking step makes use of the *reciprocal rank fusion* (RRF) algorithm, which involves combining the ranks of different search results to produce a single, unified ranking. By combining ranks from different queries, we pull the most relevant documents to the top of the final list. RRF is well-suited for combining results from queries that might have different scales or distributions of scores.

Let's demonstrate RAG-Fusion in code. First, we craft a prompt similar to the multi-query retrieval strategy to generate a list of queries based on the user query:

Python

```
from langchain.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

prompt_rag_fusion = ChatPromptTemplate.from_template("""You are a helpful
assistant that generates multiple search queries based on a single input
query. \n
Generate multiple search queries related to: {question} \n
Output (4 queries):""")

def parse_queries_output(message):
    return message.content.split('\n')

llm = ChatOpenAI(temperature=0)

query_gen = prompt_rag_fusion | llm | parse_queries_output
```

JavaScript

```
import {ChatPromptTemplate} from '@langchain/core/prompts';
import {ChatOpenAI} from '@langchain/openai';
import {RunnableLambda} from '@langchain/core/runnables';

const perspectivesPrompt = ChatPromptTemplate.fromTemplate(`You are a helpful
assistant that generates multiple search queries based on a single input
query. \n
```

```
Generate multiple search queries related to: {question} \n
Output (4 queries):`)
```

```
const queryGen = perspectivesPrompt.pipe(llm).pipe(message => {
  return message.content.split('\n')
})
```

Once we've generated our queries, we fetch relevant documents for each query and pass them into a function to *rerank* (that is, *reorder* according to relevancy) the final list of relevant documents.

The function `reciprocal_rank_fusion` takes a list of the search results of each query, so a list of lists of documents, where each inner list of documents is sorted by their relevance to that query. The RRF algorithm then calculates a new score for each document based on its ranks (or positions) in the different lists and sorts them to create a final reranked list.

After calculating the fused scores, the function sorts the documents in descending order of these scores to get the final reranked list, which is then returned:

Python

```
def reciprocal_rank_fusion(results: list[list], k=60):
    """reciprocal rank fusion on multiple lists of ranked documents
       and an optional parameter k used in the RRF formula
    """

    # Initialize a dictionary to hold fused scores for each document
    # Documents will be keyed by their contents to ensure uniqueness
    fused_scores = {}
    documents = {}

    # Iterate through each list of ranked documents
    for docs in results:
        # Iterate through each document in the list,
        # with its rank (position in the list)
        for rank, doc in enumerate(docs):
            # Use the document contents as the key for uniqueness
            doc_str = doc.page_content
            # If the document hasn't been seen yet,
            # - initialize score to 0
            # - save it for later
            if doc_str not in fused_scores:
                fused_scores[doc_str] = 0
                documents[doc_str] = doc
            # Update the score of the document using the RRF formula:
            # 1 / (rank + k)
            fused_scores[doc_str] += 1 / (rank + k)

    # Sort the documents based on their fused scores in descending order
```

```

# to get the final reranked results
reranked_doc_strs = sorted(
    fused_scores, key=lambda d: fused_scores[d], reverse=True
)
# retrieve the corresponding doc for each doc_str
return [
    documents[doc_str]
    for doc_str in reranked_doc_strs
]

retrieval_chain = generate_queries | retriever.batch | reciprocal_rank_fusi

```

JavaScript

```

function reciprocalRankFusion(results, k = 60) {
  // Initialize a dictionary to hold fused scores for each document
  // Documents will be keyed by their contents to ensure uniqueness
  const fusedScores = {}
  const documents = {}

  results.forEach(docs => {
    docs.forEach((doc, rank) => {
      // Use the document contents as the key for uniqueness
      const key = doc.pageContent
      // If the document hasn't been seen yet,
      // - initialize score to 0
      // - save it for later
      if (!(key in fusedScores)) {
        fusedScores[key] = 0
        documents[key] = 0
      }
      // Update the score of the document using the RRF formula:
      // 1 / (rank + k)
      fusedScores[key] += 1 / (rank + k)
    })
  })

  // Sort the documents based on their fused scores in descending order
  // to get the final reranked results
  const sorted = Object.entries(fusedScores).sort((a, b) => b[1] - a[1])
  // retrieve the corresponding doc for each key
  return sorted.map(([key]) => documents[key])
}

const retrievalChain = queryGen
  .pipe(retriever.batch.bind(retriever))
  .pipe(reciprocalRankFusion)

```

Notice that the function also takes a `k` parameter, which determines how much influence documents in each query's result sets have over the final

list of documents. A higher value indicates that lower-ranked documents have more influence.

Finally, we combine our new retrieval chain (now using RRF) with the full chain we've seen before:

Python

```
prompt = ChatPromptTemplate.from_template("""Answer the following question
on this context:

{context}

Question: {question}
""")

llm = ChatOpenAI(temperature=0)

@chain
def multi_query_qa(input):
    # fetch relevant documents
    docs = retrieval_chain.invoke(input)
    # format prompt
    formatted = prompt.invoke({"context": docs, "question": input})
    # generate answer
    answer = llm.invoke(formatted)
    return answer

multi_query_qa.invoke("""Who are some key figures in the ancient greek history
of philosophy?""")
```

JavaScript

```
const rewritePrompt = ChatPromptTemplate.fromTemplate(`Answer the following
question based on this context:

{context}

Question: {question}
`)

const llm = new ChatOpenAI({temperature: 0})

const multiQueryQa = RunnableLambda.from(async input => {
    // fetch relevant documents
    const docs = await retrievalChain.invoke(input)
    // format prompt
    const formatted = await prompt.invoke({context: docs, question: input})
    // generate answer
    const answer = await llm.invoke(formatted)
    return answer
})
```

```
})
```

```
await multiQueryQa.invoke(`Who are some key figures in the ancient greek  
history of philosophy?`)
```

RAG-Fusion's strength lies in its ability to capture the user's intended expression, navigate complex queries, and broaden the scope of retrieved documents, enabling serendipitous discovery.

Hypothetical Document Embeddings

Hypothetical Document Embeddings (HyDE) is a strategy that involves creating a hypothetical document based on the user's query, embedding the document, and retrieving relevant documents based on vector similarity.⁴ The intuition behind HyDE is that an LLM-generated hypothetical document will be more similar to the most relevant documents than the original query, as shown in [Figure 3-7](#).

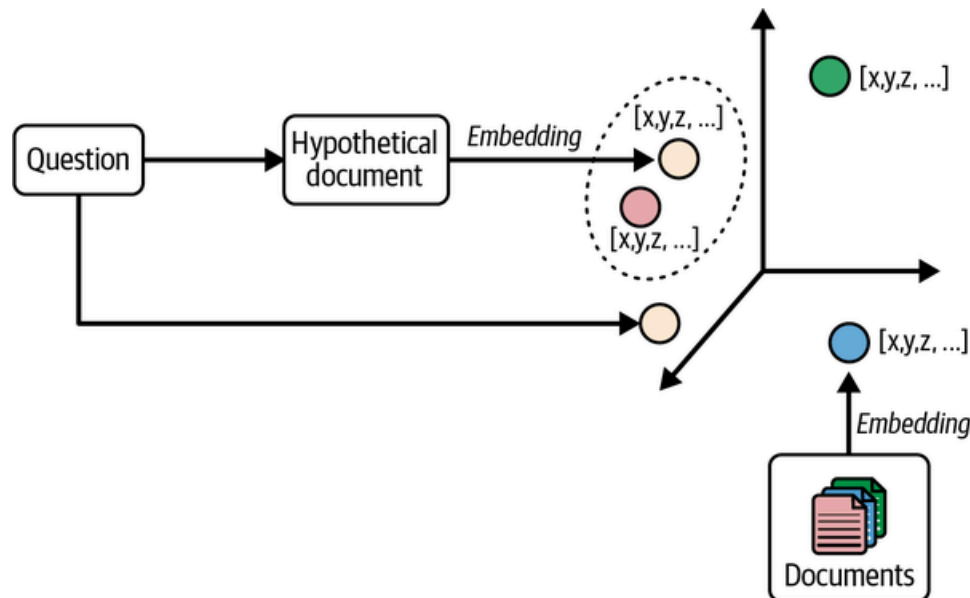


Figure 3-7. An illustration of HyDE closer in the vector space to the document embeddings than the plain query embeddings

First, define a prompt to generate a hypothetical document:

Python

```
from langchain.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain_openai import ChatOpenAI

prompt_hyde = ChatPromptTemplate.from_template("""Please write a passage to  
answer the question.\n Question: {question} \n Passage:""")

generate_doc = (
    prompt_hyde | ChatOpenAI(temperature=0) | StrOutputParser()
)
```

JavaScript

```
import {ChatOpenAI} from '@langchain/openai'
import {ChatPromptTemplate} from '@langchain/core/prompts'
import {RunnableLambda} from '@langchain/core/runnables';

const prompt = ChatPromptTemplate.fromTemplate(`Please write a passage to
  answer the question
  Question: {question}
  Passage:`)

const llm = new ChatOpenAI({temperature: 0})

const generateDoc = prompt.pipe(llm).pipe(msg => msg.content)
```

Next, we take the hypothetical document and use it as input to the `retriever`, which will generate its embedding and search for similar documents in the vector store:

Python

```
retrieval_chain = generate_doc | retriever
```

JavaScript

```
const retrievalChain = generateDoc.pipe(retriever)
```

Finally, we take the retrieved documents, pass them as context to the final prompt, and instruct the model to generate an output:

Python

```
prompt = ChatPromptTemplate.from_template("""Answer the following question
  on this context:

  {context}

  Question: {question}
  """)

llm = ChatOpenAI(temperature=0)

@chain
def qa(input):
    # fetch relevant documents from the hyde retrieval chain defined earlier
    docs = retrieval_chain.invoke(input)
    # format prompt
    formatted = prompt.invoke({"context": docs, "question": input})
```

```
# generate answer
answer = llm.invoke(formatted)
return answer

qa.invoke("""Who are some key figures in the ancient greek history of
philosophy?""")
```

JavaScript

```
const prompt = ChatPromptTemplate.fromTemplate(`Answer the following
question based on this context:

{context}

Question: {question}
`)

const llm = new ChatOpenAI({temperature: 0})

const qa = RunnableLambda.from(async input => {
  // fetch relevant documents from the hyde retrieval chain defined earlier
  const docs = await retrievalChain.invoke(input)
  // format prompt
  const formatted = await prompt.invoke({context: docs, question: input})
  // generate answer
  const answer = await llm.invoke(formatted)
  return answer
})

await qa.invoke(`Who are some key figures in the ancient greek history of
philosophy?`)
```

To recap what we covered in this section, query transformation consists of taking the user's original query and doing the following:

- Rewriting into one or more queries
- Combining the results of those queries into a single set of the most relevant results

Rewriting the query can take many forms, but it's usually done in a similar fashion: take the user's original query—a prompt you wrote—and ask an LLM to write a new query or queries. Some examples of typical changes made are:

- Removing irrelevant/unrelated text from the query.
- Grounding the query with past conversation history. For instance, to make sense of a query such as *and what about in LA*, we need to combine it with a hypothetical past question about the weather in SF, to arrive at a useful query such as *weather in LA*.

- Casting a wider net for relevant documents by also fetching documents for related queries.
- Decomposing a complex question into multiple, simpler questions and then including results for all of them in the final prompt to generate an answer.

The right rewriting strategy to use will depend on your use case.

Now that we've covered the main query transformation strategies, let's discuss the second major question to answer in order to build a robust RAG system: How do we route queries to retrieve relevant data from multiple data sources?

Query Routing

Although using a single vector store is useful, the required data may live in a variety of data sources, including relational databases or other vector stores.

For example, you may have two vector stores: one for LangChain Python documentation and another for LangChain JS documentation. Given a user's question, we would like to *route* the query to the appropriate inferred data source to retrieve relevant docs. *Query routing* is a strategy used to forward a user's query to the relevant data source.

Logical Routing

In *logical routing*, we give the LLM knowledge of the various data sources at our disposal and then let the LLM reason which data source to apply based on the user's query, as shown in [Figure 3-8](#).

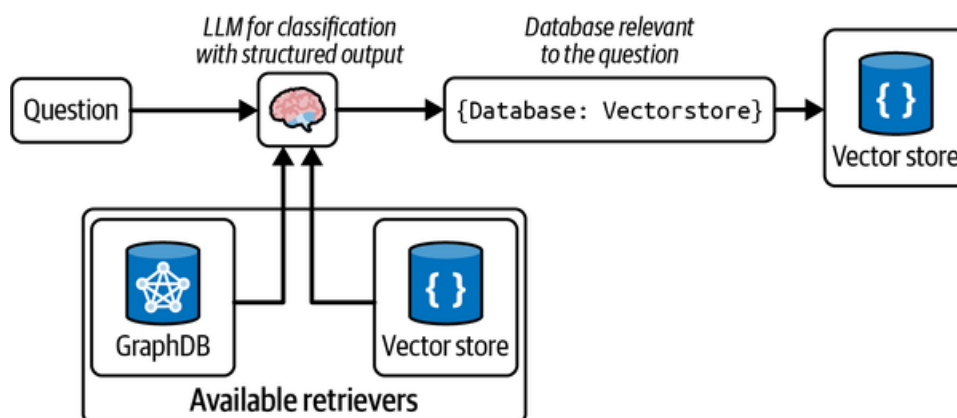


Figure 3-8. Query routing to relevant data sources

In order to achieve this, we make use of function-calling models like GPT-3.5 Turbo to help classify each query into one of the available routes. A *function call* involves defining a schema that the model can use to generate arguments of a function based on the query. This enables us to gener-

ate structured outputs that can be used to run other functions. The following Python code defines the schema for our router based on three docs for different languages:

Python

```
from typing import Literal

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.pydantic_v1 import BaseModel, Field
from langchain_openai import ChatOpenAI

# Data model
class RouteQuery(BaseModel):
    """Route a user query to the most relevant datasource."""

    datasource: Literal["python_docs", "js_docs"] = Field(
        ...,
        description="Given a user question, choose which datasource would "
        "most relevant for answering their question",
    )

# LLM with function call
llm = ChatOpenAI(model="gpt-3.5-turbo", temperature=0)
structured_llm = llm.with_structured_output(RouteQuery)

# Prompt
system = """You are an expert at routing a user question to the appropriate
source.

Based on the programming language the question is referring to, route it to
relevant data source."""

prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system),
        ("human", "{question}"),
    ]
)

# Define router
router = prompt | structured_llm
```

JavaScript

```
import { ChatOpenAI } from "@langchain/openai";
import { z } from "zod";

const routeQuery = z.object({
  datasource: z.enum(["python_docs", "js_docs"]).describe(`Given a user
  question, choose which datasource would be most relevant for answering
```

```

        their question`),
    })).describe("Route a user query to the most relevant datasource.")

const llm = new ChatOpenAI({model: "gpt-3.5-turbo", temperature: 0})
const structuredLlm = llm.withStructuredOutput(routeQuery, {name: "RouteQue

const prompt = ChatPromptTemplate.fromMessages([
  ['system', `You are an expert at routing a user question to the appropriate
    data source.

Based on the programming language the question is referring to, route it to
    the relevant data source.`],
  ['human', '{question}']]
])

const router = prompt.pipe(structuredLlm)

```

Now we invoke the LLM to extract the data source based on the predefined schema:

Python

```

question = """Why doesn't the following code work:

from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages(["human", "speak in {language}"])
prompt.invoke("french")
"""

result = router.invoke({"question": question})

result.datasource
# "python_docs"

```

JavaScript

```

const question = `Why doesn't the following code work:

from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages(["human", "speak in {language}"])
prompt.invoke("french")
`

await router.invoke({ question })

```

The output:

```
{
  datasource: "python_docs"
}
```

Notice how the LLM produced JSON output, conforming to the schema we defined earlier. This will be useful in many other tasks.

Once we've extracted the relevant data source, we can pass the value into another function to execute additional logic as required:

Python

```
def choose_route(result):
    if "python_docs" in result.datasource.lower():
        ### Logic here
        return "chain for python_docs"
    else:
        ### Logic here
        return "chain for js_docs"

full_chain = router | RunnableLambda(choose_route)
```

JavaScript

```
function chooseRoute(result) {
  if (result.datasource.toLowerCase().includes('python_docs')) {
    return 'chain for python_docs';
  } else {
    return 'chain for js_docs';
  }
}

const fullChain = router.pipe(chooseRoute)
```

Notice how we don't do an exact string comparison but instead first turn the generated output to lowercase, and then do a substring match. This makes our chain more resilient to the LLM going off script and producing output that doesn't quite conform to the schema we asked for.

TIP

Resilience to the random nature of LLM outputs is an important theme to keep in mind when building your LLM applications.

Logical routing is most suitable when you have a defined list of data sources from which relevant data can be retrieved and utilized by the

LLM to generate an accurate output. These can range from vector stores to databases and even APIs.

Semantic Routing

Unlike logical routing, *semantic routing* involves embedding various prompts that represent various data sources alongside the user's query and then performing vector similarity search to retrieve the most similar prompt. [Figure 3-9](#) illustrates.

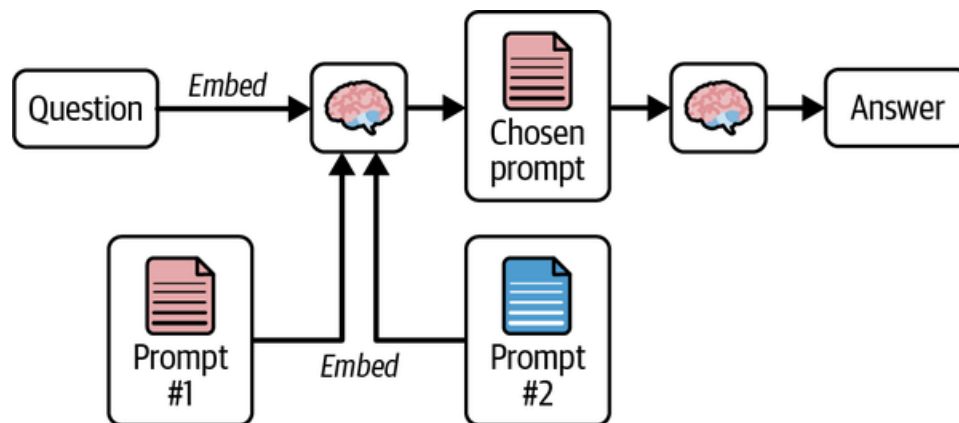


Figure 3-9. Semantic routing to improve the accuracy of retrieved documents

The following is an example of semantic routing:

Python

```
from langchain.utils.math import cosine_similarity
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import PromptTemplate
from langchain_core.runnables import chain
from langchain_openai import ChatOpenAI, OpenAIEmbeddings

# Two prompts
physics_template = """You are a very smart physics professor. You are great
    answering questions about physics in a concise and easy-to-understand m
    When you don't know the answer to a question, you admit that you don't

Here is a question:
{query}"""

math_template = """You are a very good mathematician. You are great at answ
    math questions. You are so good because you are able to break down hard
    problems into their component parts, answer the component parts, and th
    put them together to answer the broader question.

Here is a question:
{query}"""

# Embed prompts
embeddings = OpenAIEmbeddings()
```

```

prompt_templates = [physics_template, math_template]
prompt_embeddings = embeddings.embed_documents(prompt_templates)

# Route question to prompt
@chain
def prompt_router(query):
    # Embed question
    query_embedding = embeddings.embed_query(query)
    # Compute similarity
    similarity = cosine_similarity([query_embedding], prompt_embeddings)[0]
    # Pick the prompt most similar to the input question
    most_similar = prompt_templates[similarity.argmax()]
    return PromptTemplate.from_template(most_similar)

semantic_router = (
    prompt_router
    | ChatOpenAI()
    | StrOutputParser()
)

print(semantic_router.invoke("What's a black hole"))

```

JavaScript

```

import {cosineSimilarity} from '@langchain/core/utils/math'
import {ChatOpenAI, OpenAIEmbeddings} from '@langchain/openai'
import {PromptTemplate} from '@langchain/core/prompts'
import {RunnableLambda} from '@langchain/core/runnables';

const physicsTemplate = `You are a very smart physics professor. You are gr
    at answering questions about physics in a concise and easy-to-understand
    manner. When you don't know the answer to a question, you admit that you
    don't know.

Here is a question:
{query}`

const mathTemplate = `You are a very good mathematician. You are great at
    answering math questions. You are so good because you are able to break d
    hard problems into their component parts, answer the component parts, and
    then put them together to answer the broader question.

Here is a question:
{query}`

const embeddings = new OpenAIEmbeddings()

const promptTemplates = [physicsTemplate, mathTemplate]
const promptEmbeddings = await embeddings.embedDocuments(promptTemplates)

const promptRouter = RunnableLambda.from(query => {
    // Embed question
    const queryEmbedding = await embeddings.embedQuery(query)

```

```
// Compute similarity
const similarities = cosineSimilarity([queryEmbedding], promptEmbeddings)
// Pick the prompt most similar to the input question
const mostSimilar = similarities[0] > similarities[1]
  ? promptTemplates[0]
  : promptTemplates[1]
return PromptTemplate.fromTemplate(mostSimilar)
})

const semanticRouter = promptRouter.pipe(new ChatOpenAI())

await semanticRouter.invoke("What's a black hole")
```

Now that you’ve seen how to route a user’s query to the relevant data source, let’s discuss the third major question when building a robust RAG system: “How do we transform natural language to the query language of the target data source?”

Query Construction

As discussed earlier, RAG is an effective strategy to embed and retrieve relevant unstructured data from a vector store based on a query. But most data available for use in production apps is structured and typically stored in relational databases. In addition, unstructured data embedded in a vector store also contains structured metadata that possesses important information.

Query construction is the process of transforming a natural language query into the query language of the database or data source you are interacting with. See [Figure 3-10](#).

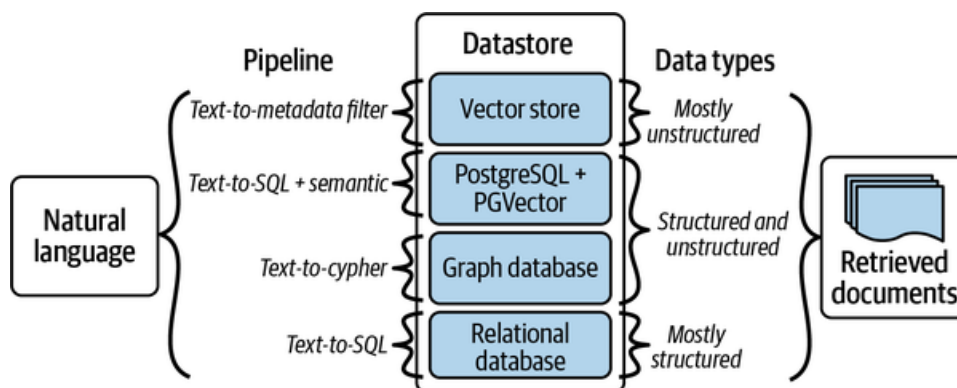


Figure 3-10. Illustration of query languages for various data sources

For example, consider the query: *What are movies about aliens in the year 1980?* This question contains an unstructured topic that can be retrieved via embeddings (*aliens*), but it also contains potential structured components (*year == 1980*).

The following sections dive deeper into the various forms of query construction.

Text-to-Metadata Filter

Most vector stores provide the ability to limit your vector search based on metadata. During the embedding process, we can attach metadata key-value pairs to vectors in an index and then later specify filter expressions when you query the index.

LangChain provides a `SelfQueryRetriever` that abstracts this logic and makes it easier to translate natural language queries into structured queries for various data sources. The self-querying utilizes an LLM to extract and execute the relevant metadata filters based on a user's query and predefined metadata schema:

Python

```
from langchain.chains.query_constructor.base import AttributeInfo
from langchain.retrievers.self_query.base import SelfQueryRetriever
from langchain_openai import ChatOpenAI

fields = [
    AttributeInfo(
        name="genre",
        description="The genre of the movie",
        type="string or list[string]",
    ),
    AttributeInfo(
        name="year",
        description="The year the movie was released",
        type="integer",
    ),
    AttributeInfo(
        name="director",
        description="The name of the movie director",
        type="string",
    ),
    AttributeInfo(
        name="rating", description="A 1-10 rating for the movie", type="float"
    ),
]
description = "Brief summary of a movie"

llm = ChatOpenAI(temperature=0)

retriever = SelfQueryRetriever.from_llm(
    llm, db, description, fields,
)
```



```
print(retriever.invoke(
    "What's a highly rated (above 8.5) science fiction film?"))
```

JavaScript

```
import { ChatOpenAI } from "@langchain/openai";
import { SelfQueryRetriever } from "langchain/retrievers/self_query";
import { FunctionalTranslator } from "@langchain/core/structured_query";

/**
 * First, we define the attributes we want to be able to query on.
 * in this case, we want to be able to query on the genre, year, director,
 * rating, and length of the movie.
 * We also provide a description of each attribute and the type of the attr
 * This is used to generate the query prompts.
 */
const fields = [
  {
    name: "genre",
    description: "The genre of the movie",
    type: "string or array of strings",
  },
  {
    name: "year",
    description: "The year the movie was released",
    type: "number",
  },
  {
    name: "director",
    description: "The director of the movie",
    type: "string",
  },
  {
    name: "rating",
    description: "The rating of the movie (1-10)",
    type: "number",
  },
  {
    name: "length",
    description: "The length of the movie in minutes",
    type: "number",
  },
];

const description = "Brief summary of a movie";

const llm = new ChatOpenAI();
const attributeInfos = fields.map((field) => new AttributeInfo(field.name,
    field.description, field.type));

const selfQueryRetriever = SelfQueryRetriever.fromLLM({
  llm,
  db,
  description,
```

```

attributeInfo: attributeInfos,
/**
 * We need to use a translator that translates the queries into a
 * filter format that the vector store can understand. LangChain provides
 * here.
 */
structuredQueryTranslator: new FunctionalTranslator(),
});

await selfQueryRetriever.invoke(
  "What's a highly rated (above 8.5) science fiction film?"
);

```

This results in a retriever that will take a user query, and split it into:

- A filter to apply on the metadata of each document first
- A query to use for semantic search on the documents

To do this, we have to describe which fields the metadata of our documents contain; that description will be included in the prompt. The retriever will then do the following:

1. Send the query generation prompt to the LLM.
2. Parse metadata filter and rewritten search query from the LLM output.
3. Convert the metadata filter generated by the LLM to the format appropriate for our vector store.
4. Issue a similarity search against the vector store, filtered to only match documents whose metadata passes the generated filter.

Text-to-SQL

SQL and relational databases are important sources of structured data, but they don't interact directly with natural language. Although we can simply use the LLM to translate a user's query to SQL queries, there is little margin for error.

Here are some useful strategies for effective text to SQL translations:

Database description

To ground SQL queries, an LLM must be provided with an accurate description of the database. One common text-to-SQL prompt employs an idea reported in this paper and others: provide the LLM with a `CREATE TABLE` description for each table, including column names and types.⁵ We can also provide a few (for instance, three) example rows from the table.

Few-shot examples

Feeding the prompt with few-shot examples of question-query matches can improve the query generation accuracy. This can be achieved by simply appending standard static examples in the prompt to guide the agent on how it should build queries based on questions.

See [Figure 3-11](#) for a visual of the process.

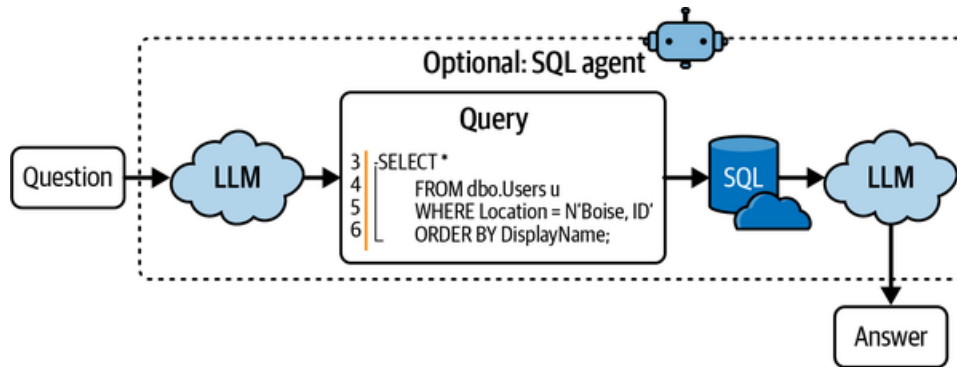


Figure 3-11. A user's query transformed to a SQL query

Here's a full code example:

Python

```
from langchain_community.tools import QuerySQLDatabaseTool
from langchain_community.utilities import SQLDatabase
from langchain.chains import create_sql_query_chain
from langchain_openai import ChatOpenAI

# replace this with the connection details of your db
db = SQLDatabase.from_uri("sqlite:///Chinook.db")
llm = ChatOpenAI(model="gpt-4", temperature=0)

# convert question to sql query
write_query = create_sql_query_chain(llm, db)

# Execute SQL query
execute_query = QuerySQLDatabaseTool(db=db)

# combined
chain = write_query | execute_query

# invoke the chain
chain.invoke('How many employees are there?');
```

JavaScript

```
import { ChatOpenAI } from "@langchain/openai";
import { createSqlQueryChain } from "langchain/chains/sql_db";
import { SqlDatabase } from "langchain/sql_db";
import { DataSource } from "typeorm";
```

```
import { QuerySqlTool } from "langchain/tools/sql";

const datasource = new DataSource({
  type: "sqlite",
  database: "./Chinook.db", // here should be the details of your database
});
const db = await SqlDatabase.fromDataSourceParams({
  appDataSource: datasource,
});
const llm = new ChatOpenAI({ model: "gpt-4", temperature: 0 });

// convert question to sql query
const writeQuery = await createSqlQueryChain({ llm, db, dialect: "sqlite" });

// execute query
const executeQuery = new QuerySqlTool(db);

// combined
const chain = writeQuery.pipe(executeQuery);

// invoke the chain
await chain.invoke('How many employees are there?');
```

We first convert the user's query to a SQL query appropriate to the dialect of our database. Then we execute that query on our database. Note that executing arbitrary SQL queries on your database generated by an LLM from user input is dangerous in a production application. To use these ideas in production, you need to consider a number of security measures to reduce the risk of unintended queries being run in your database. Here are some examples:

- Run the queries on your database with a user with read-only permissions.
- The database user running the queries should have access only to the tables you wish to make available for querying.
- Add a time-out to the queries run by this application; this would ensure that even if an expensive query is generated, it is canceled before taking up too many of your database resources.

This is not an exhaustive list of security considerations. Security of LLM applications is an area currently in development, with more security measures being added to the recommendations as new vulnerabilities are discovered.

Summary

This chapter discussed various state-of-the-art strategies to efficiently retrieve the most relevant documents based on a user's query and synthe-

size them with your prompt to aid the LLM to generate an accurate, up-to-date output.

As discussed, a robust, production-ready RAG system requires a wide range of effective strategies that can execute query transformation, query construction, routing, and indexing optimization.

Query transformation enables your AI app to transform an ambiguous or malformed user query into a representative query that's optimal for retrieval. Query construction enables your AI app to convert the user's query into the syntax of the query language of the database or data source where structured data lives. Routing enables your AI app to dynamically route the user's query to retrieve relevant information from the relevant data source.

In [Chapter 4](#), we'll build on this knowledge to add memory to your AI chatbot so that it can remember and learn from each interaction. This will enable users to “chat” with the application in multiturn conversations like ChatGPT.

- ¹ Patrick Lewis et al., [“Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”](#), arXiv, April 12, 2021.
- ² Xinbei Ma et al., [“Query Rewriting for Retrieval-Augmented Large Language Models”](#), arXiv, October 23, 2023. Research commissioned by Microsoft Research Asia.
- ³ Zackary Rackauckas, [“RAG-Fusion: A New Take on Retrieval-Augmented Generation”](#), arXiv, February 21, 2024. From the *International Journal on Natural Language Computing*, vol. 13, no. 1 (February 2024).
- ⁴ Luyu Gao et al., [“Precise Zero-Shot Dense Retrieval Without Relevance Labels”](#), arXiv, December 20, 2022.
- ⁵ Nitarshan Rajkumar et al., [“Evaluating the Text-to-SQL Capabilities of Large Language Models”](#), arXiv, March 15, 2022.